

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

ARC 赛后讲解

2024.11.17



Competition Host

Abstraction and Reasoning
Corpus



Prizes & Awards

\$1,100,000

Awards Points & Medals

Participation

16,354 Entrants

1,642 Participants

1,430 Teams

17,789 Submissions

Tags

Artificial Intelligence

Custom Metric

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 如何上手比赛

➤ 比赛基础

- Python基础：基础语法
- 开源包使用：pandas、numpy、matplotlib
- 机器学习包：scikit-learn
- 深度学习包：tensorflow、pytorch

➤ 任务拆解

- 题目分析：充分理解赛题含义和比赛评价目标
- 熟悉数据：理解数据的特征和含义，对数据进行统计学分析
- 特征工程：充分理解数据后，构造特征，为下一步建模做准备
- 模型建立：建立机器学习和深度学习模型

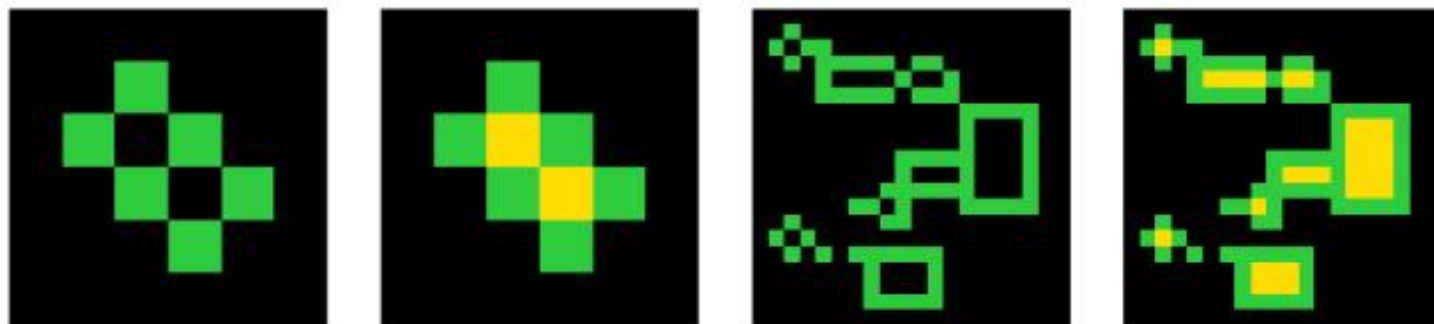
ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 比赛描述

这项比赛的目标是让计算机仅通过少量例子学习复杂的抽象任务。

目前的机器学习技术对数据需求量大且较为脆弱，它们只能理解之前见过的模式。使用当前的方法，算法可以通过接触大量数据来获得新技能，但能够广泛泛化到多种任务的认知能力依然较弱。



ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 比赛注意事项

这是一场**代码竞赛**参赛作品必须通过笔记本完成，必须满足以下条件：

- CPU Notebook $\leq$ 12小时运行时间
- GPU Notebook $\leq$ 12小时运行时间
- 禁止上网
- 允许免费和公开的外部数据，包括预先训练的模型
- 提交文件必须命名为submission.json

Notebook options ^

ACCELERATOR

GPU P100

Quota: 

LANGUAGE

Python

PERSISTENCE

No persistence

ENVIRONMENT

Pin to original environment

You won't get new packages, but your code is less likely to break. What is a notebook environment? ⓘ

INTERNET

☒ Internet on

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 数据概览

Data Explorer

4.2 MB

```
{i} arc-agi_evaluation_challenges
{i} arc-agi_evaluation_solutions.js
{i} arc-agi_test_challenges.json
{i} arc-agi_training_challenges.js
{i} arc-agi_training_solutions.jsor
{i} sample_submission.json
```

Summary

▸ 📁 6 files

小样本学习任务

Task files

The information is stored in two files:

- **arc-agi_training-challenges.json**: contains input/output pairs that demonstrate reasoning pattern to be applied to the "test" input for each task. This file and the corresponding **solutions** file can be used as training for your models.
- **arc-agi_training-solutions.json**: contains the corresponding task "test" outputs (ground truth).
- **arc-agi_evaluation-challenges.json**: contains input/output pairs that demonstrate reasoning pattern to be applied to the "test" input for each task. This file and the corresponding **solutions** file can be used as validation data for your models.
- **arc-agi_evaluation-solutions.json**: contains the corresponding task "test" outputs (ground truth).
- **arc-agi_test-challenges.json**: this file contains the tasks that will be used for the leaderboard evaluation, and contains "train" input/output pairs as well as the "test" input for each task. Your task is to predict the "test" output. **Note:** The file shown on this page is a placeholder using tasks from **arc-agi_evaluation-challenges.json**. When you submit your notebook to be rerun, this file is swapped with the actual test challenges.
- **sample_submission.json**: a submission file in the correct format

Each task contains a dictionary with two fields:

- **"train"** : demonstration input/output pairs. It is a list of "pairs" (typically 3 pairs).
- **"test"** : test input - your model should predict the output.

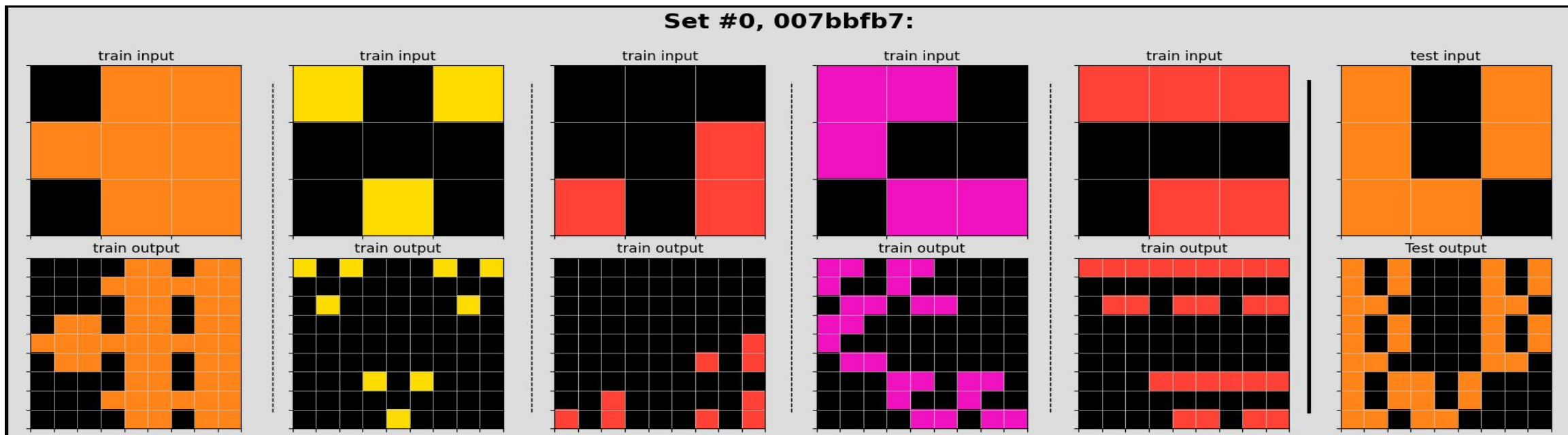
A "pair" is a dictionary with two fields:

- **"input"** : the input "grid" for the pair.
- **"output"** : the output "grid" for the pair.

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 数据概览

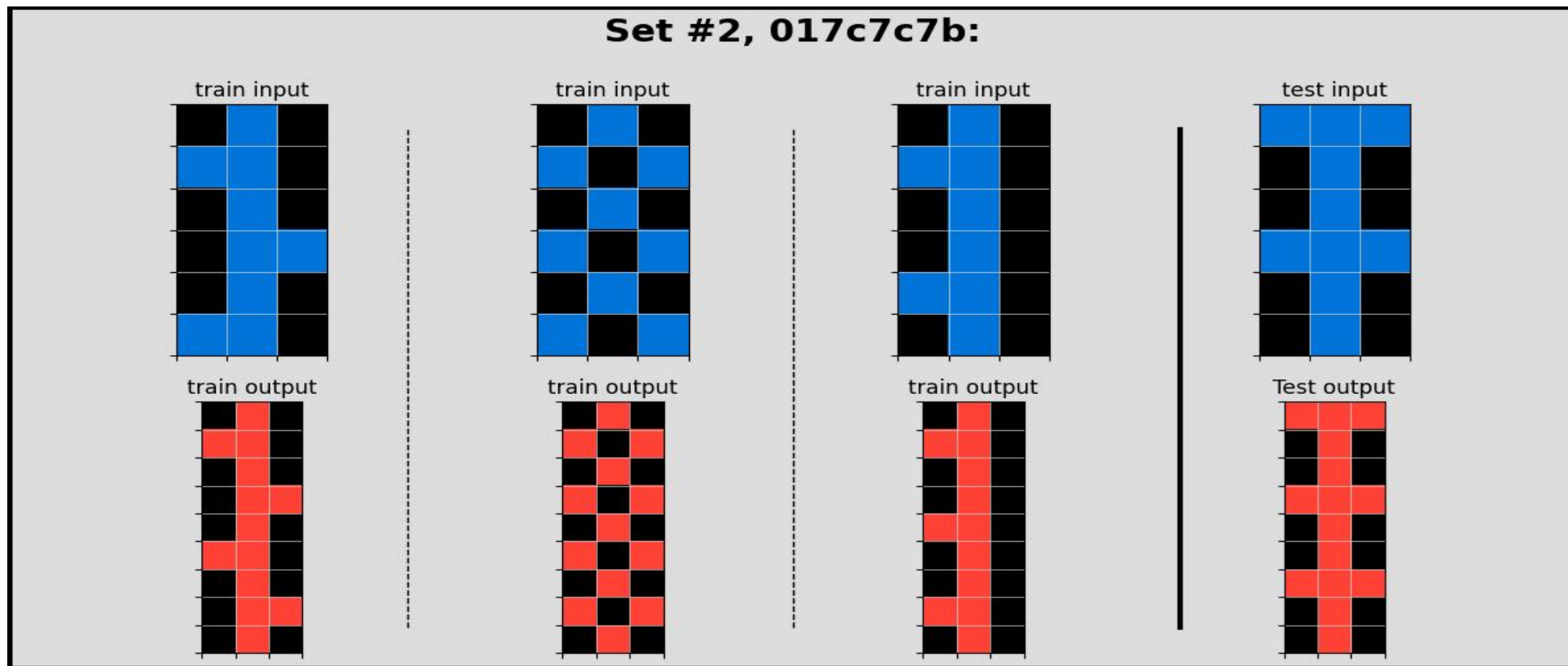


一个图像 重复3*3次

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 数据概览

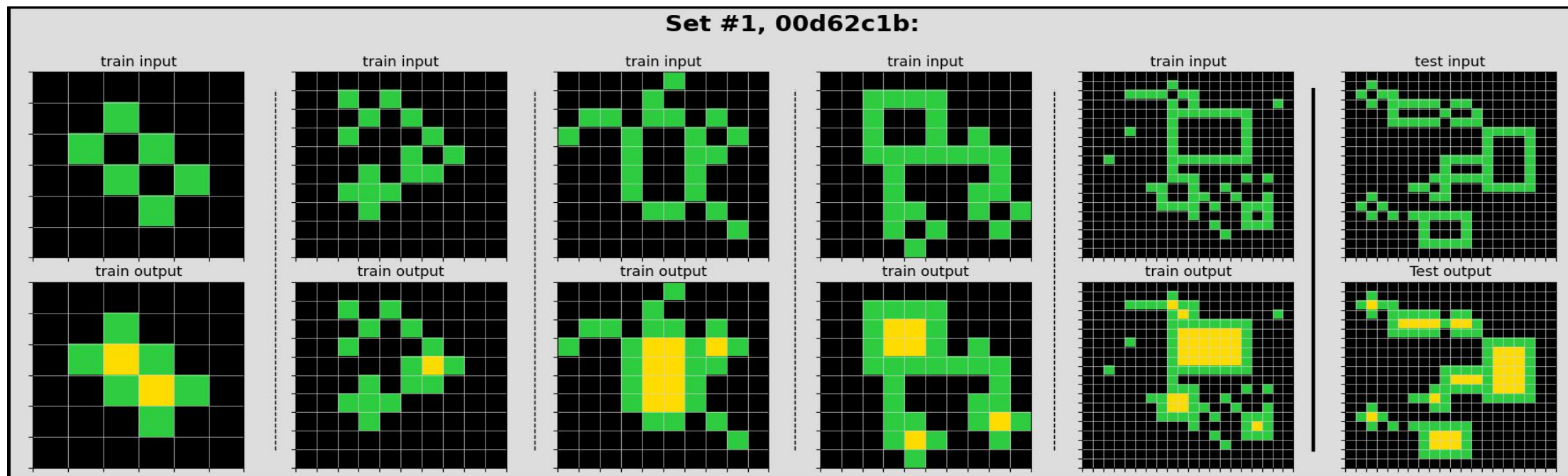


重复两次 换色

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 数据概览

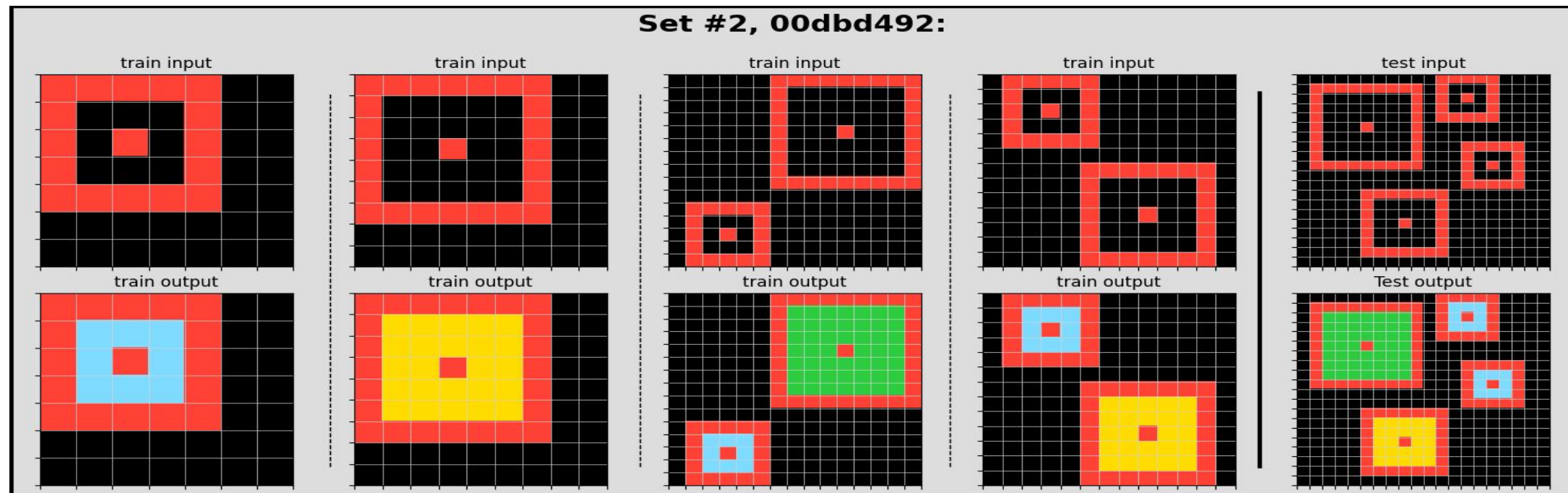


闭环 上色

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 数据概览



相同形状上色

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 评价指标

This competition evaluates submissions on the percentage of correct predictions. For each task, you should predict exactly 2 outputs for every test input grid contained in the task. (Tasks can have more than one test input that needs a predicted output.) Each task test output has one ground truth. For a given task output, any of the 2 predicted outputs matches the ground truth exactly, you score 1 for that task test output, otherwise 0. The final score is the sum averaged of the highest score per task output divided by the total number of task test outputs.

■ 提交文件

```
{
  "00576224": [{"attempt_1": [[0, 0], [0, 0]], "attempt_2": [[0, 0], [0, 0]]}],
  "009d5c81": [{"attempt_1": [[0, 0], [0, 0]], "attempt_2": [[0, 0], [0, 0]]}],
  "12997ef3": [{"attempt_1": [[0, 0], [0, 0]], "attempt_2": [[0, 0], [0, 0]]},
               {"attempt_1": [[0, 0], [0, 0]], "attempt_2": [[0, 0], [0, 0]]}],
  ...
}
```

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 思路：Domain Specific Language DSL方法

- 1 将数组切割成矩阵块作为对象
- 2 旋转、填充、翻转、移动、上色等变换，for loop test N次
- 3 和标准答案对比，看是否一致
- 4 如果能完美匹配，合并结果

变换^深度：变换 几十种 深度2 3 4

python

 复制代码

```
def converter(inp, objects, c) -> objects:  
    # inp: 数据集中的原始输入  
    # objects: 处理后的输入数据表示，带有一些元数据，比如矩形信息  
    # c: 在训练和测试之间共享的上下文对象，例如只出现在输出中的颜色，输出形状信息
```

python

 复制代码

```
def merger(objects, c):  
    # objects: 处理后的输入数据  
    # c: 上下文对象
```

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 变换方法

读取task

```
def get_objects(task):  
    xs, ys = [], []  
    for obj in task['train']:  
        xs.append(np.array(obj['input']))  
        ys.append(np.array(obj['output']))  
    return xs, ys
```

局部图像

```
def find_sub(matrix, sub, ignore = None):  
    positions = []  
    mask = sub != ignore  
    sub_ = sub[mask]  
    for x in range(matrix.shape[0]-sub.shape[0]+1):  
        for y in range(matrix.shape[1]-sub.shape[1]+1):  
            if np.array_equal(matrix[x:x+sub.shape[0], y:y+sub.shape[1]][mask], sub_):  
                positions.append((x,y,x+sub.shape[0],y+sub.shape[1]))  
    return positions
```


ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 变换方法

确定颜色

```
def find_colors(g):
    colors = np.unique(g)
    counts = len(colors)
    for cnt in range(counts, 1, -1):
        q_colors = np.full(cnt, -1, int)
        for c in colors:
            pts = np.where(g == c)
            s = set([(x+y)%cnt for x, y in zip(*pts)])
            if len(s) > 1:
                continue
            index = s.pop()
            q_colors[index] = c

    if -1 not in q_colors:
        return q_colors
    return None
```

是否重复

```
def has_repeating(g, ignore=0):
    size0b = int(.6 * g.shape[0])
    size1b = int(.6 * g.shape[1])
    t = np.full((g.shape[0]+2*size0b, g.shape[1]+2*size1b), -1)
    t[size0b:-size0b, size1b:-size1b] = g
    t[t==ignore] = -1
    for size0 in range(2, size0b+1):
        for size1 in range(2, size1b+1):
            for shift0 in range(size0):
                for shift1 in range(size1):
                    pattern = t[size0b+shift0:size0b+shift0+size0, size1b+shift1:size1b+shift1+size1].copy()

                    found = True
                    for d0 in range(size0b+shift0-size0, t.shape[0]-size0, size0):
                        for d1 in range(size1b+shift1-size1, t.shape[1]-size1, size1):
                            test = t[d0:d0+size0, d1:d1+size1]
                            mask = (test != -1) & (pattern != -1)
                            if np.array_equal(test[mask], pattern[mask]):
                                ind = test != -1
                                pattern[ind] = test[ind]
                            else:
                                found = False
                                break
                    if found:
                        return shift0, shift1, pattern

    return None
```

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 变换方法

图像截断

```
def trim_matrix(x):
    if len(np.unique(x)) == 1:
        return x
    for c in np.unique(x):
        xs,ys = np.where(x!=c)
        xmin,ymin,xmax,ymax = min(xs),min(ys),max(xs)+1,max(ys)+1
        if xmin > 0 or ymin > 0 or xmax < x.shape[0] or ymax < x.shape[1]:
            return x[xmin:xmax,ymin:ymax]
    return x

def trim_matrix_box(g, mask=None):
    if mask is None:
        mask = np.unique(g)
    if len(np.unique(g)) == 1:
        return None
    for c in mask:
        xs,ys = np.where(g!=c)
        xmin,ymin,xmax,ymax = min(xs),min(ys),max(xs)+1,max(ys)+1
        if xmin > 0 or ymin > 0 or xmax < g.shape[0] or ymax < g.shape[1]:
            return (xmin,ymin,xmax,ymax)
    return None
```

旋转变换

```
def get_all_transforms():
    return [
        lambda x: roll_color(x),
        lambda x: np.roll(x, -1, axis=0),
        lambda x: np.roll(x, 1, axis=0),
        lambda x: np.roll(x, -1, axis=1),
        lambda x: np.roll(x, 1, axis=1),
        lambda x: np.rot90(x.T, k=1),
        lambda x: np.rot90(x.T, k=2),
        lambda x: np.rot90(x.T, k=3),
        lambda x: np.rot90(x.T, k=4),
        lambda x: np.rot90(x, k=1),
        lambda x: np.rot90(x, k=2),
        lambda x: np.rot90(x, k=3),
        lambda x: x,
    ]
```


ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

■ 变换方法

图像截断

```
def trim_matrix(x):
    if len(np.unique(x)) == 1:
        return x
    for c in np.unique(x):
        xs,ys = np.where(x!=c)
        xmin,ymin,xmax,ymax = min(xs),min(ys),max(xs)+1,max(ys)+1
        if xmin > 0 or ymin > 0 or xmax < x.shape[0] or ymax < x.shape[1]:
            return x[xmin:xmax,ymin:ymax]
    return x

def trim_matrix_box(g, mask=None):
    if mask is None:
        mask = np.unique(g)
    if len(np.unique(g)) == 1:
        return None
    for c in mask:
        xs,ys = np.where(g!=c)
        xmin,ymin,xmax,ymax = min(xs),min(ys),max(xs)+1,max(ys)+1
        if xmin > 0 or ymin > 0 or xmax < g.shape[0] or ymax < g.shape[1]:
            return (xmin,ymin,xmax,ymax)
    return None
```

旋转变换

```
def get_all_transforms():
    return [
        lambda x: roll_color(x),
        lambda x: np.roll(x, -1, axis=0),
        lambda x: np.roll(x, 1, axis=0),
        lambda x: np.roll(x, -1, axis=1),
        lambda x: np.roll(x, 1, axis=1),
        lambda x: np.rot90(x.T, k=1),
        lambda x: np.rot90(x.T, k=2),
        lambda x: np.rot90(x.T, k=3),
        lambda x: np.rot90(x.T, k=4),
        lambda x: np.rot90(x, k=1),
        lambda x: np.rot90(x, k=2),
        lambda x: np.rot90(x, k=3),
        lambda x: x,
    ]
```

ARC Prize 2024

Create an AI capable of solving reasoning tasks it has never seen before

答疑环节